

Lecture 18

Structured Concurrency

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Pizza Robots!

```
export class Robot {
  public readonly robot_id: string;

  public constructor(robot_id: string) {
    this.robot_id = robot_id;
  }

  // Asynchronously adds topping to pizza.
  // The returned promise resolves when the
  // topping has been added. It occasionally
  // rejects if the robot ruined the pizza.
  async addToppingAsync(
    pizza: Pizza, topping: string
  ): Promise<void> { ... }
}

const ROBOTS = [ new Robot("Robot 1"), ... ];

// Pool of currently available robots
const availableRobots: Robot[] = [...ROBOTS];
```

```
// Waitlist of robot requests (promise resolvers)
const waitlist: Array<(r: Robot)=>void> = [];

// Request a robot. Resolves immediately if one is
// available, otherwise add the resolver function
// to the waitlist.
export function acquireRobot(): Promise<Robot> {
  const robot = availableRobots.shift();
  if (robot) { return Promise.resolve(robot); }
  else {
    return new Promise(res => waitlist.push(res));
  }
}

// If there are requests waiting, call the resolver
// for the next promise. Otherwise, return it to
// the pool of available robots.
export function releaseRobot(robot: Robot): void {
  const nextWaiter = robotWaitlist.shift();
  if (nextWaiter) { nextWaiter(robot); }
  else { availableRobots.push(robot); }
}
```

A Robot is a **resource** that must
be acquired and released.

Recall: Async/Await

- A function declared as `async` is allowed to use the `await` expression.
- When called with a promise, `await` essentially suspends execution of the function until the promise **resolves**. Then it resumes as a continuation.

```
async function addCheeseThenPepperoniThenCelebrate(pizza: Pizza): Promise<void> {  
  const robot = await acquireRobot();  
  await robot.addToppingAsync(pizza, "cheese");  
  await robot.addToppingAsync(pizza, "pepperoni");  
  console.log("I did it!");  
}
```

- If the promise **rejects**, `await` instead throws an exception.

```
export class Robot {  
  // Asynchronously adds topping to pizza.  
  // The returned promise resolves when the  
  // topping has been added. It occasionally  
  // rejects if the robot ruined the pizza.  
  async addToppingAsync(  
    pizza: Pizza, topping: string  
  ): Promise<void> { }  
}
```

```
// Waitlist of robot requests  
const waitlist: Array<(r: Robot)=>void> = [];  
  
// Request a robot. The returned promise  
// resolves with the robot once available.  
export function acquireRobot(): Promise<Robot>;  
  
// Release the robot so it can do other stuff.  
export function releaseRobot(robot: Robot): void;
```

What could go wrong with this function?

```
async function addAllToppings(pizza: Pizza): Promise<void> {  
  const recipe = TOPPINGS[pizza.pizzaKind];  
  const robot = await acquireRobot();  
  await Promise.all(recipe.map(topping => robot.addToppingAsync(pizza, topping)));  
  releaseRobot(robot);  
}
```

If one of the topping promises rejects, `Promise.all()` rejects. The `await` converts this to an exception, and `releaseRobot()` never runs!

```
export class Robot {  
  // Asynchronously adds topping to pizza.  
  // The returned promise resolves when the  
  // topping has been added. It occasionally  
  // rejects if the robot ruined the pizza.  
  async addToppingAsync(  
    pizza: Pizza, topping: string  
  ): Promise<void> { }  
}
```

```
// Waitlist of robot requests  
const waitlist: Array<(r: Robot)=>void> = [];  
  
// Request a robot. The returned promise  
// resolves with the robot once available.  
export function acquireRobot(): Promise<Robot>;  
  
// Release the robot so it can do other stuff.  
export function releaseRobot(robot: Robot): void;
```

What could go wrong with this function?

```
async function addAllToppings(pizza: Pizza): Promise<void> {  
  const recipe = TOPPINGS[pizza.pizzaKind];  
  const robot = await acquireRobot();  
  try {  
    await Promise.all(recipe.map(topping => robot.addToppingAsync(pizza, topping)));  
  } finally {  
    releaseRobot(robot);  
  }  
}
```

If one of the topping promises rejects, the exception gets thrown as before. The finally block ensures that `releaseRobot()` runs, but the other topping tasks keep going after the robot was released!

AbortController

- AbortController is a standard utility for cancelation.
- **Parent/Owner:**
 - Create an AbortController instance.
 - Pass to child tasks, observer subscriptions, etc.
 - Call `.abort()` to cancel tasks, subscriptions, timeouts, etc.
- **Child:**
 - Poll `.signal.aborted` while working **OR** listen for an 'abort' event.
 - Do your best to cancel work if aborted, then resolve.



Claude's attempt at using an AbortController

```
async function addAllToppings(pizza: Pizza): Promise<void> {
  const recipe = TOPPINGS[pizza.pizzaKind];
  const robot = await acquireRobot();
  const controller = new AbortController();

  try {
    await Promise.all(recipe.map(async (topping) => {
      // Check if already aborted before starting
      if (controller.signal.aborted) { throw new Error("Aborted"); }

      await robot.addToppingAsync(pizza, topping);

      // Check if aborted after completing (too late to undo, but stop further work)
      if (controller.signal.aborted) { throw new Error("Aborted"); }
    })));
  } catch (err) {
    controller.abort(); // Signal other tasks to stop
    throw err;          // Re-throw the original error
  } finally {
    releaseRobot(robot);
  }
}
```

Corrected Implementation

```
async function addAllToppings(pizza: Pizza): Promise<void> {
  const recipe = TOPPINGS[pizza.pizzaKind];
  const robot = await acquireRobot();
  const controller = new AbortController();

  const tasks = recipe.map(topping =>
    robot.addToppingAsyncCancellable(pizza, topping, controller.signal)
  );

  try {
    await Promise.all(tasks); // Fail-fast on first error
  } catch (err) {
    controller.abort();      // cancel all tasks
    throw err;
  } finally {
    await Promise.allSettled(tasks); // Wait for all to finish before cleanup
    releaseRobot(robot);
  }
}
```



Structured Concurrency

- **Structured programming** restricts control flow to well-defined recognizable patterns like `if`, `for`, `while`.
- **Structured concurrency** restricts concurrency to well-defined, recognizable patterns. Usually these rules:
 - A task is owned by a lexical block scope or “nursery” object.
 - A task may not execute past the end of its owner.
 - No background tasks executing indefinitely.
 - All “split” concurrent tasks come back to a single point.
 - An owner waits on all subtasks before cleaning up and moving on.
 - Automatic error propagation and cancelation.



Corrected Implementation

```
async function addAllToppingsStructured(pizza: Pizza): Promise<void> {  
    const recipe = TOPPINGS[pizza.pizzaKind];  
    const robot = await acquireRobot();  
    const controller = new AbortController();  
  
    try {  
        // TaskGroup owns each of the subtasks. Would also support a .cancelAll() fn.  
        await using nursery = new TaskGroup(recipe.map(topping =>  
            robot.addToppingAsyncCancellable(pizza, topping, controller.signal)  
        ));  
  
        // Automatically handles errors, cancellation, waiting for subtasks.  
        // If there were failures, they're re-thrown/propagated after all of that.  
    } finally {  
        releaseRobot(robot);  
    }  
}
```

Structured Concurrency

Check out these implementations:

- **Typescript**: Effection Library (generator-based)
- **Python**: `asyncio.TaskGroup`
- **Java**: Project Loom, `StructuredTaskScope`
- **Kotlin**: coroutines and scopes
- **Swift**: `async/await` with structured task groups